Practice Exam #1 (150 pts)

SDS 270 — Programming for Data Science in R, Spring 2026

Name: _____

Exam Rules

- You have **75 minutes** to complete this exam.
- No computers, cellphones, or other internet-connected devices. This exam is closed-note and to be completed individually.
- Metadata for the dataset specified in the questions can be found at the back of the exam.
- Give your best effort and attempt every question, laying out your thought-process for questions/parts that may be ambiguous for you. Partial credit may be awarded.

Honor Pledge

Smith College expects all students to be honest and committed to the principles of academic and intellectual integrity in their preparation and submission of coursework and examinations. All submitted work of any kind must be the original work of the student who must cite all the sources used in its preparation.

By signing my name on this exam, I confirm that I have completed this exam with integrity and honesty. I did not use any non-approved materials, electronic devices, or other aids to assist me on the exam.

Signature: _____

This page left intentionally blank.

Q1: Data Preparation (40 pts)

(a) (8 pts) Highlight the main differences between atomic vector and list objects.

AV	List
- composed of one type	- composed of diff. types / complex types
- less space	- uses more space
- access elements w/ [ ]	- access elements w/ [[ ]]

(b) (8 pts) Let the `Area` and `Perimeter` be the first two columns in the dataset. Describe two ways we can access the areas and perimeters of all observations with an `Area` between 30,000 and 40,000 (inclusive).

data [Area >= 30000 & Area <= 40000, 1:2]

data |> select (Area, Perimeter) |> filter (Area >= 30000 & Area <= 40000)

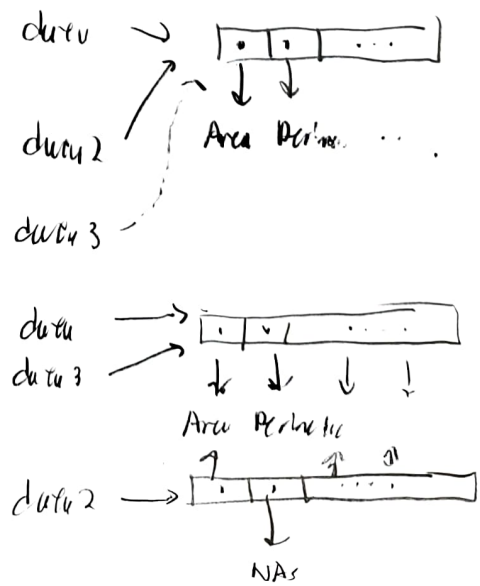
(c) (9 pts) Describe how memory is allocated for `data2` and `data3` when we run the following code at each of the checkpoints (labeled `#CP _`).

```
data2 <- data
# CP 1
data3 <- data2
# CP 2
data2[[2]] <- rep(NA, nrow(data2))
# CP 3
```

CP1 → all bindings in both data frames refer to same columns/memory

CP2 → all bindings are the same as before

CP3 → shallow copy of data2, binding to 2nd column is changed



(d) (8 pts) Write pseudo-code/code that would print the summary of the `AspectRatio` for each group of the `Class` column in our `data` object.

tapply (data$AspectRatio, data$Class, summary)

(e) (7 pts) Describe what would happen in the moment and with the `data` if we ran the following command:

```
data$ShapeTogether <- ifelse(data$ShapeFactor1 < 0.007, 1, "yes")
```

↳ creates new column named ShapeTogether
↳ composed of "1" and "yes" due to coercion
↳ "1" if ShapeFactor1[i] < 0.007
"yes" otherwise

↳ done w/ vectorized if

Q2: Sorting (40 pts)

In this question, you will be using the `order()` and `sort()` functions to rearrange and/or sort the observations in the dataset. Here are summaries of the two functions with their arguments:

- `sort(x, decreasing)`: Sorts a vector/factor into ascending or descending order.
  - `x`: vector/factor
  - `decreasing`: defaults as `FALSE`, can also be `TRUE`. Dictates which direction vector/factor is sorted.
- `order(x, decreasing)`: Returns an atomic vector specifying the indices of the elements in ascending or descending order.
  - `x`: vector/factor
  - `decreasing`: defaults as `FALSE`, can also be `TRUE`. Dictates which direction vector/factor is sorted.

As an example, if we have `x <- c(2.5, 3.7, 9.6, 4.3)`, then:

- `sort(x)` would return 2.5, 3.7, 4.3, 9.6
- `order(x)` would return 1, 2, 4, 3

Use this information to answer the following parts.

(a) (8 pts) Using pseudocode or code, describe how we can use one/both of the aforementioned functions to retrieve the bottom 5 observations from the `data` based on its perimeter.

head (data [order (data$perimeter),] , 5)

(b) (8 pts) Using pseudocode or code, describe how we can use one/both of the aforementioned functions to retrieve the highest 100 values of the eccentricity vector.

sort (data$Eccentricity , decreasing = T) [1 : 100]

(c) (8 pts) Suppose we want to store the bottom 5 observations based on each non-character variable in a pre-allocated list through the use of a for-loop. Critique the code below and spot areas in which problems may occur.

```
5bottom <- vector(mode = "list", length = ncol(data))
ind = 1
for (i in 1:ncol(data)){
  if (typeof(data[i, ]) != "character"){
    5bottom[[ind]] <- # correct code used from part (a) for ind
  }
}
```

need to increase ind

reference column, not row

variable name

(d) (8 pts) Discuss how we can use a functional to perform the same task as seen in part (c).

```
bottom5Funct <- function(x, data) {
    return (data[order(x), ][1:5, ])
map(data, bottom5Funct, data)
```

(e) (8 pts) Address the memory implications of the process seen in parts (c) and (d), including potential alternatives or fixes.

1 vs. 2

more time ✓

more memory used ✓

more room for typos/errors ✓

Q3: Evaluating Changes in Script (35 pts)

(a) (24 pts) Suppose we create code that defines new objects in R and creates a new function to run through a calculation:

```
within2SDs <- function(x){
  meanX <- mean(x)  - built R
  sdX <- sd(x)
  within2 <- mean(x > meanX - 2 * sdX & x < meanX + 2 * sdX)
  return(within2)
}

x <- c(1, 8, 5, 2, 9)
```

For each sub-part (6 pts each), describe the code's behavior when running the following chunk in R:

(i)
```
mean <- function(x){(min(x) + max(x)) / 2}
within2SDs(x)
```

- creates mean() function inside environment
- uses new mean() function when running within2SDs function
- gets method/unexpected result for x

(ii)
```
sdX <- 10
within2SDs(x)
```

- sdx is assigned 10
- sdx from function is used for within2SDs calculation on x

(iii)
```
y <- c(2, 2, 2, 2, 2)
within2SDs(y)
```

- y is assigned atomic vector of 2s
- y is used in within2SDs function

(iv)
```
within2SDs()
```

- error → no default value

(b) (11 pts) Rewrite the `within2SDs` function such that we can access additional arguments in the `mean()` function.

```
within2SDs <- function (x, ...) {
    meanx <- mean(x, ...)
    sdx <- sd(x)
    within2 <- mean(x > meanx - 2 * sdx & x > meanx + 2 * sdx)
    return (within2)
}
```

Q4: Building A Whole Script (35 pts)

Sketch out a script (of code/pseudocode and comments) that accomplishes the following:

I want to create an object in R that houses the following information for each group of `Class`:

- Percentage of areas that are below 30,000 or above 60,000
- A summary of the compactness variable
- A scatter plot of area vs. perimeter

In this sketch, specify what data/coding structures you will use and how they will accomplish the task. Consider appropriate ways to accomplish these tasks.

```
q4 <- vector (mode = "list", length = length(unique(data$ class)))

forClass <- function (x) {
    percArea <- mean (x $ Area < 30000 | x $ Area > 40000)
    summ <- summary (x $ Compactness)
    plot (x$Area, x$ Perimeter), plotOut <- recordplot ()
    return (list (perc = percArea,
                  summary = summ,
                  plot = plotOut))

tapply (data, data$ Class, forClass)
```